LAB MANUAL

COM682

Faculty of Computing, Engineering and The Built Environment

COM682 – Lab 03

Cosmos Database

Contents

1. What is Cosmos DB?

Azure Cosmos DB is a NoSQL Database. Azure Cosmos DB is Microsoft, a multi-model Database service "for managing data at planet-scale".

In this lab, you create and manage an Azure Cosmos DB SQL API account from the Azure portal and Visual Studio Code with a Python app cloned from GitHub. Azure Cosmos DB is a multi-model database service that lets you quickly create and query document, table, key-value, and graph databases with global distribution and horizontal scale capabilities.

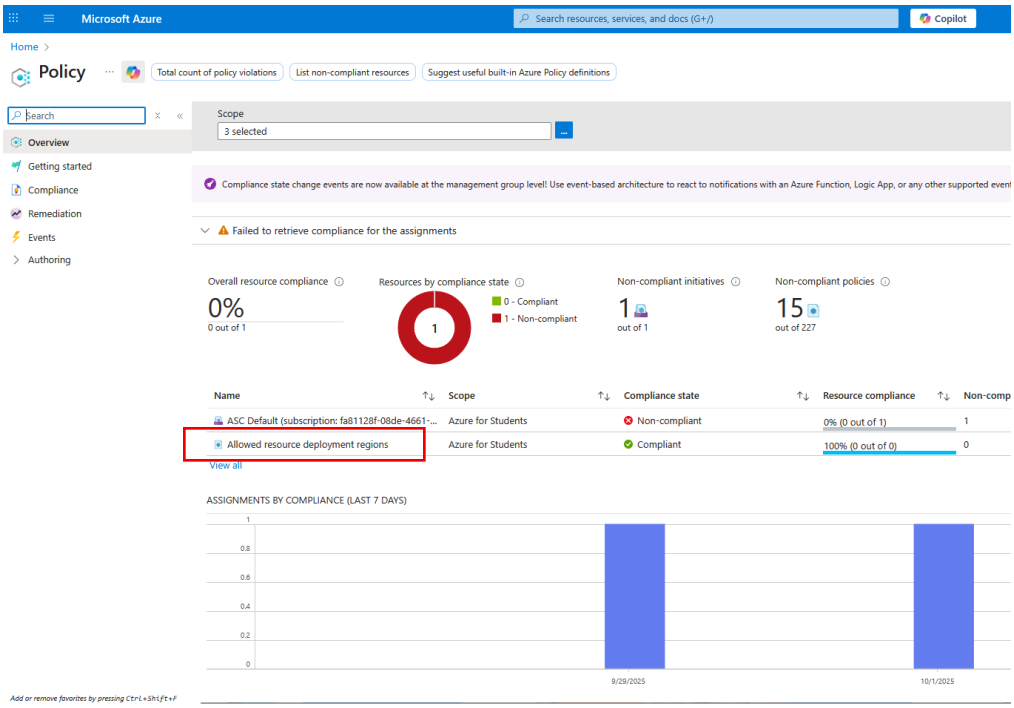
2. Prerequisites

- A Cosmos DB Account. You options are:
- Within an Azure active subscription:
- Create an Azure free account or use your existing subscription
- Visual Studio Monthly Credits
- Azure Cosmos DB Free Tier
- Without an Azure active subscription:
- Try Azure Cosmos DB for free, a tests environment that lasts for 30 days.
- Azure Cosmos DB Emulator
- Python 2.7 or 3.6+, with the python executable in your PATH.
- Visual Studio Code.
- The Python extension for Visual Studio Code.
- Git.
- Azure Cosmos DB SQL API SDK for Python

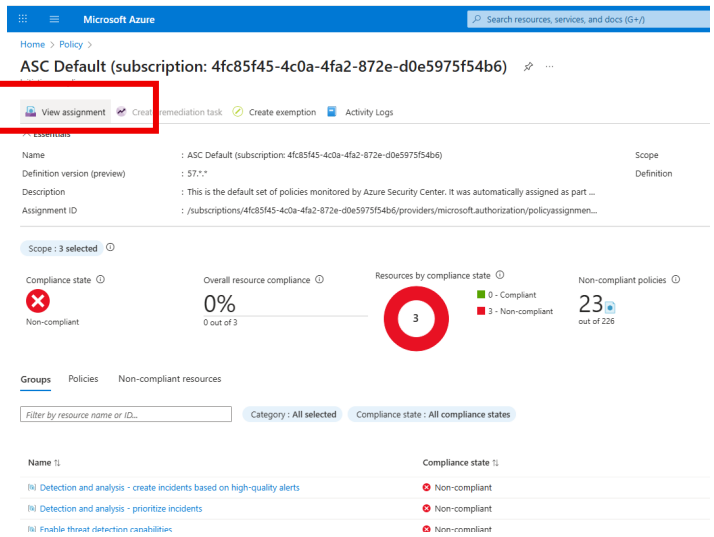
3. Find the region where you can create a resource

[Home - Microsoft Azure]

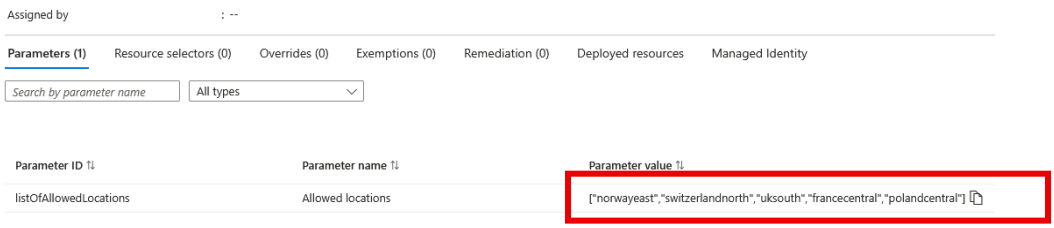
From the Azure portal menu or the **Home page**, search and select **Policy**.



- Click on 'Allowed resource Deplaoyment regions'
    - Next page, click on View Assignment



- You can see list of allowed locations.



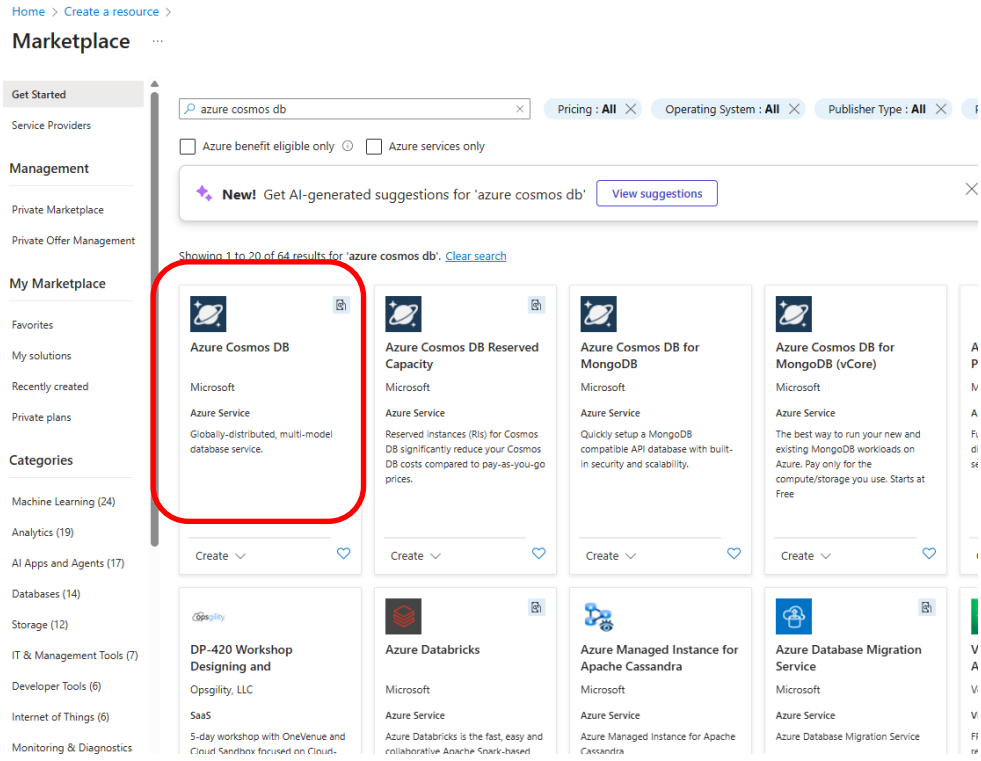
*Note this location as you would need it for creating Cosmos DB.

4. Create a database account

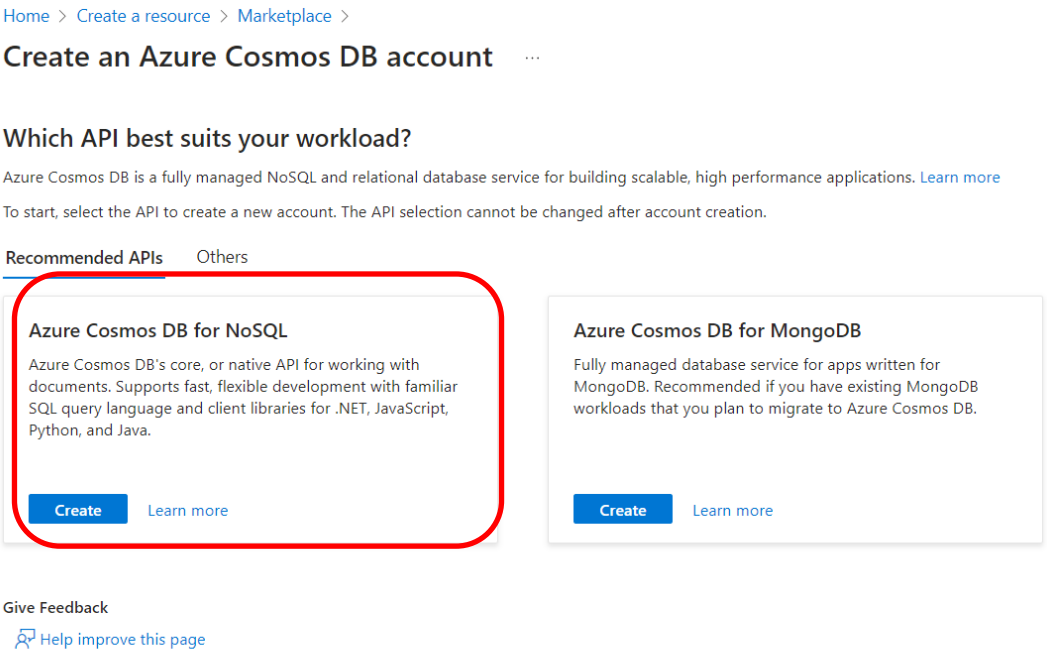
1. From the Azure portal menu or the **Home page**, select **Create a resource**.

   Home - Microsoft Azure

   On the **New** page, search for and select **Azure Cosmos DB**.



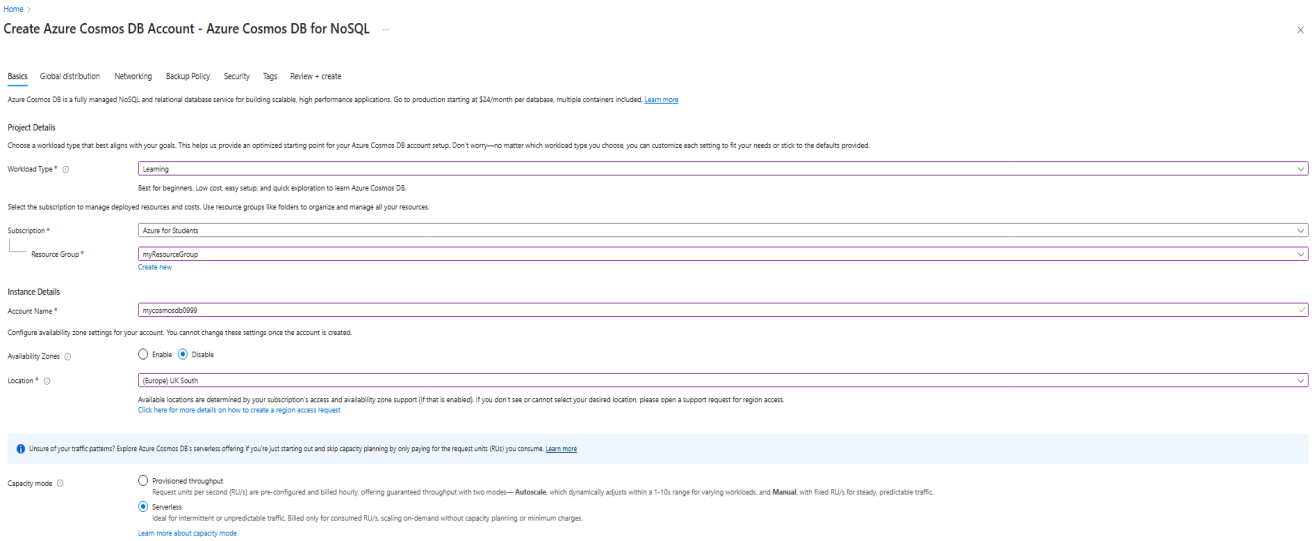
2. On the **Azure Cosmos DB** page, select **Create**.
3. Select **Azure Cosmos DB for NoSQL**



4. In the **Create Azure Cosmos DB Account - Azure Cosmos DB for NoSQL** page, enter the basic settings for the new Azure Cosmos account.

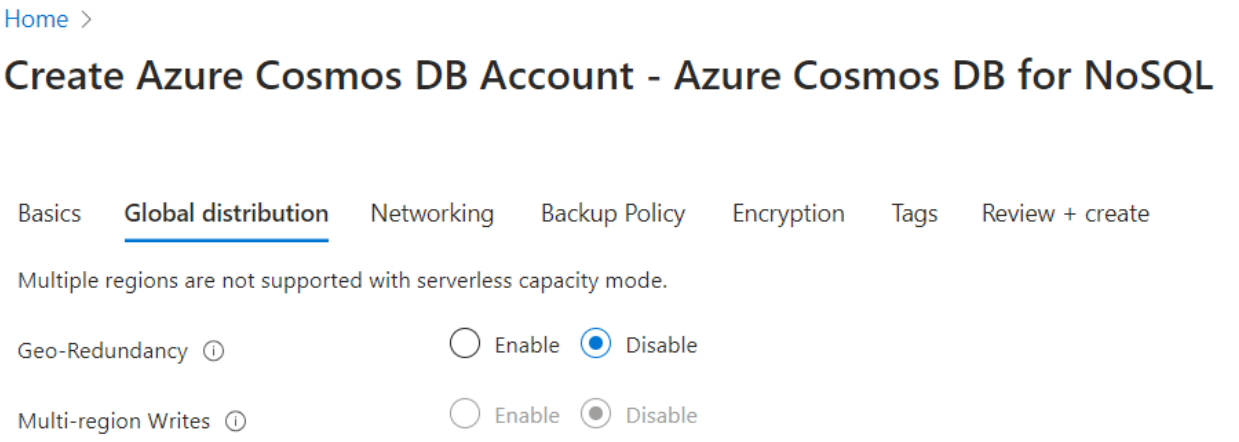
Setting	**Value**	**Description**
Workload Type	Learning	Select the Workload Type Learning.
Subscription	Subscription name	Select the Azure subscription that you want to use for this Azure Cosmos account. **(Azure for Students)**
Resource Group	Resource group name	Select a resource group, or select **Create new**, then enter a unique name for the new resource group. *CosmosDBResource*
Account Name	A unique name	Enter a name to identify your Azure Cosmos account. Because *documents.azure.com* is appended to the name that you provide to create your URI, use a unique name.

		The name can only contain lowercase letters, numbers, and the hyphen (-) character. It must be between 3-44 characters in length. *For example : cosmosaccount123*
Availability Zones		Disable
Location	The region closest to your users	Select a geographic location to host your Azure Cosmos DB account. Use the location that is closest to your users to give them the fastest access to the data. **Default = UK South (you know the location from earlier step by searching 'policy' as above)**
Capacity mode	Serverless	Select **Serverless** to create an account in serverless mode.



5. In the Global Distribution tab, configure the following details. You can leave the default values for the purpose of this quickstart:

Setting	Value	Description
Geo-Redundancy	Disable	Enable or disable global distribution on your account by pairing your region with a pair region. You can add more regions to your account later.
Multi-region Writes	Disable (By Default)	Multi-region writes capability allows you to take advantage of the provisioned throughput for your databases and containers across the globe.

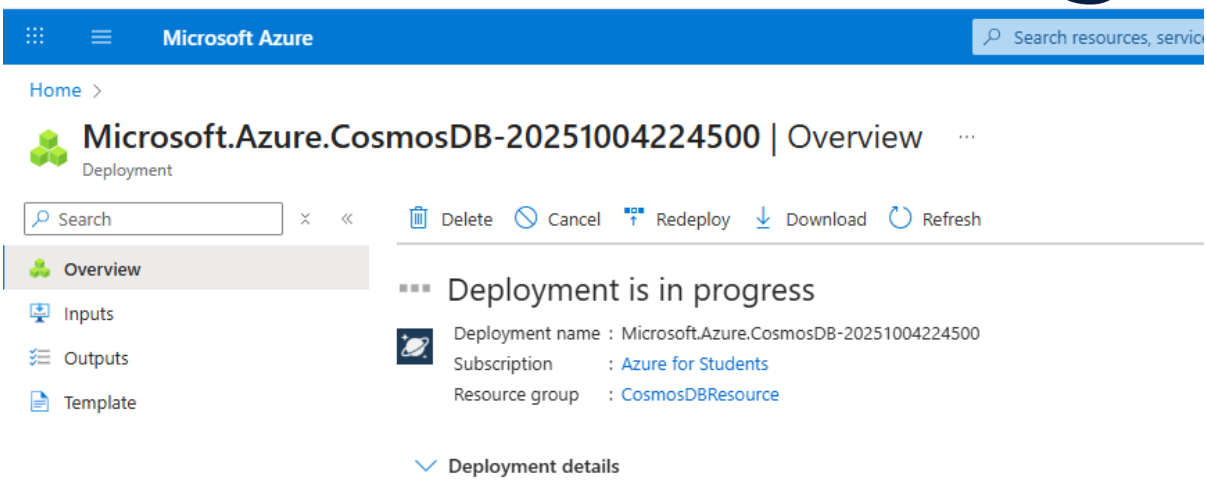


6. Optionally you can configure additional details in the following tabs:

- **Networking** – Select **All Networks**
- **Backup Policy** - Configure either periodic or continuous backup policy. You can choose Periodic default selected for this lab.
- **Encryption** - Use either service-managed key or a customer-managed key. You can choose default for this lab.
- **Tags** - Tags are name/value pairs that enable you to categorize resources and view consolidated billing by applying the same tag to multiple resources and resource groups.

- Select **Review + create**.

7. Review the account settings, and then select **Create**. It takes a few minutes to create the account. Wait for the portal page to display **Your deployment is complete**.

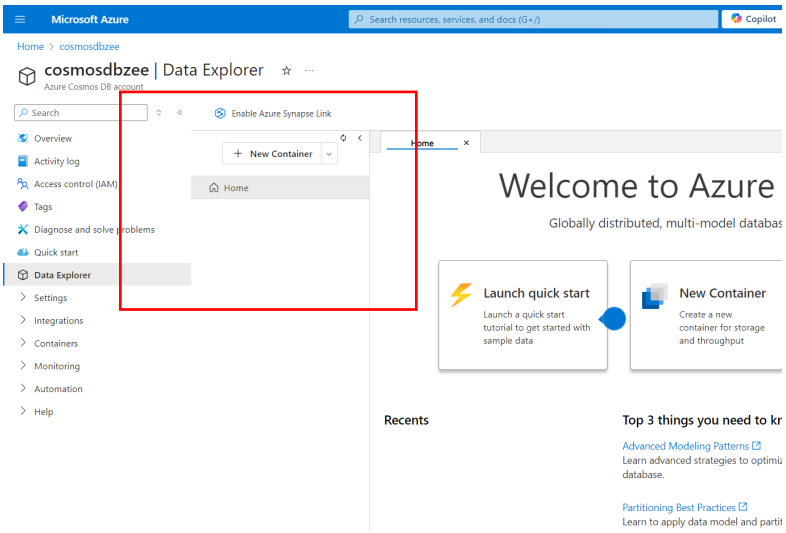


8. Once completed, Select **Go to resource** to go to the Azure Cosmos DB account page.

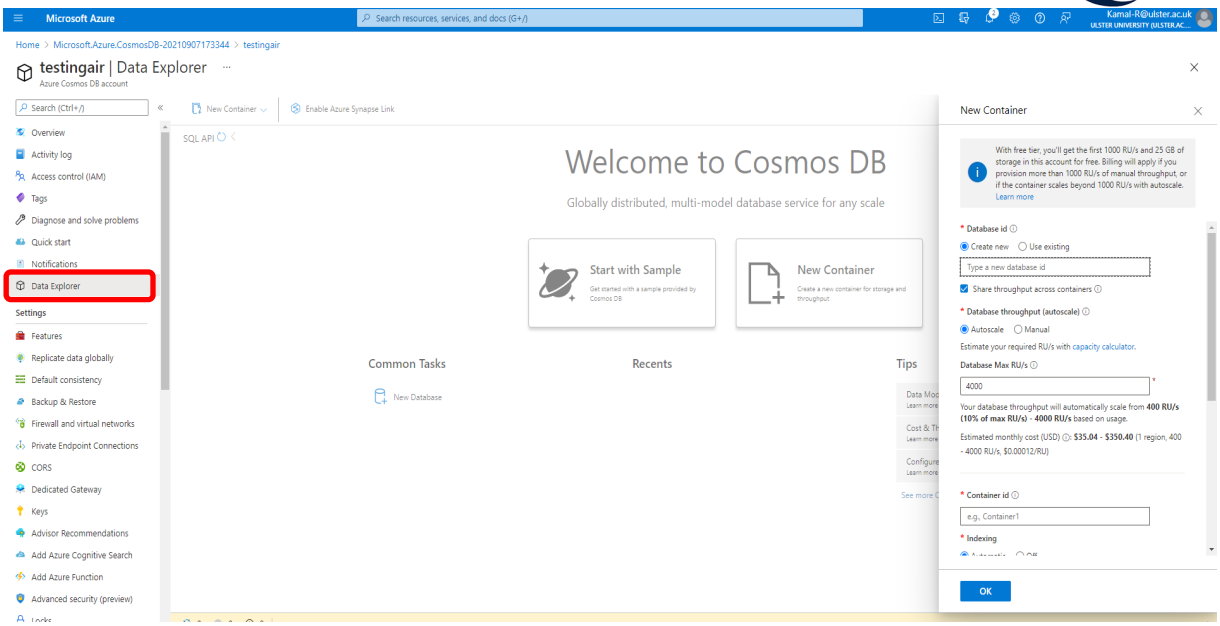
Add a container

You can now use the Data Explorer tool in the Azure portal to create a database and container.

1. Select **Data Explorer** > **New Container**.



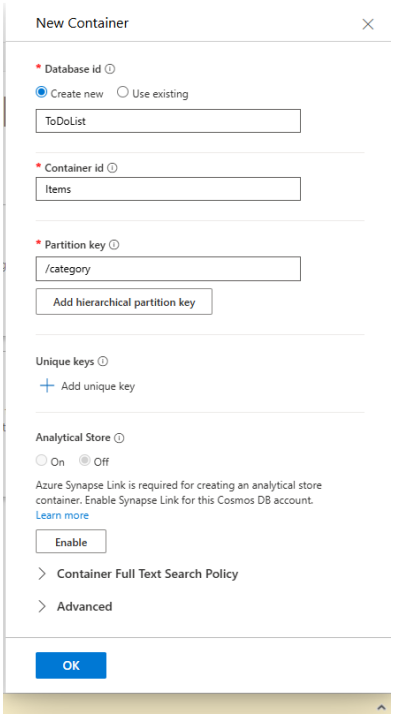
2. The **Add Container** area is displayed on the far right, or you can find it from Data explorer.



In the Add container page, enter the settings for the new container.

Setting	Suggested value	Description
Database ID	Create new: 'ToDoList'	Database names must contain from 1 through 255 characters, and they cannot contain /, \\, #, ? or a trailing space.
Container ID	Items	Enter *Items* as the name for your new container. Container IDs have the same character requirements as database names.
Partition key	/category	The Sample described in this article uses */category* as the partition key.

Don't add **Unique keys** or turn on the **Analytical store** for this example. Unique keys let you add a layer of data integrity to the Database by ensuring the uniqueness of one or more values per partition key. For more information, see Unique keys in Azure Cosmos DB. The analytical store is used to enable large-scale analytics against operational data without any impact on your transactional workloads.

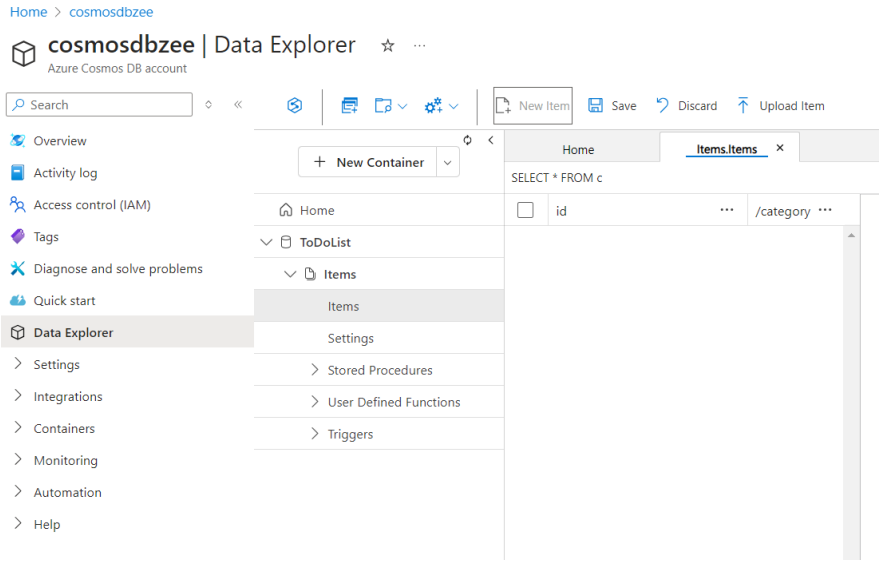


Select OK. The Data Explorer displays the new Database and container.

Add sample data

You can now add data to your new container using Data Explorer.

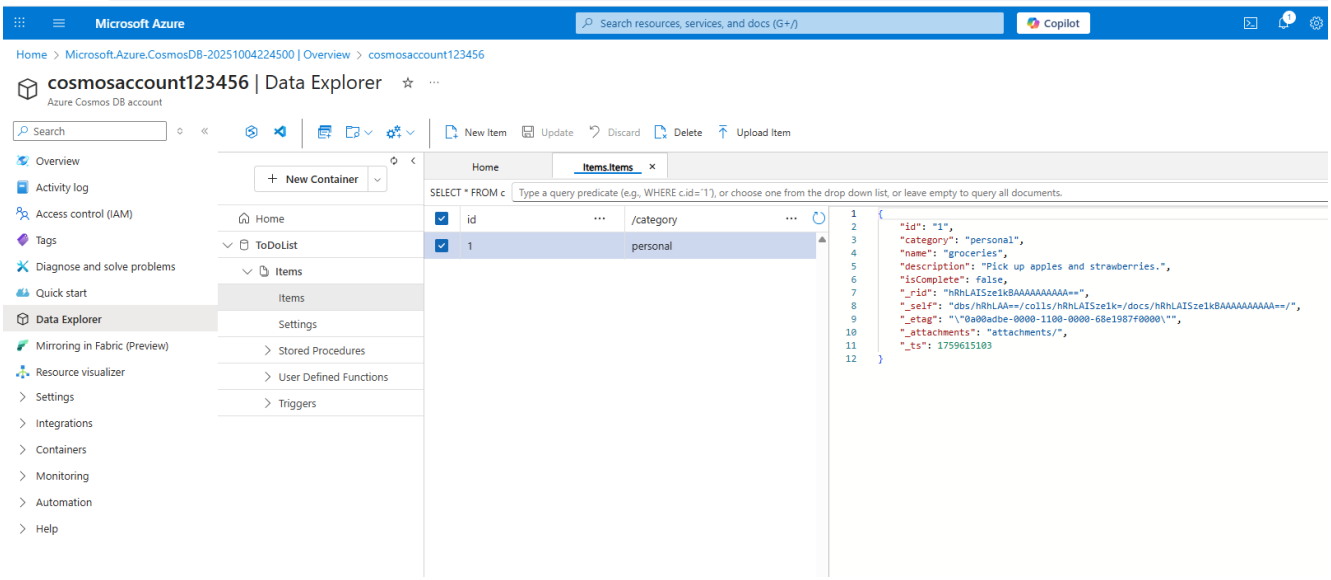
1. From the **Data Explorer**, expand the **ToDoList** database, expand the **Items** container. Select **Items**, and then select **New Item**.



2. Now add a document to the container with the following structure.

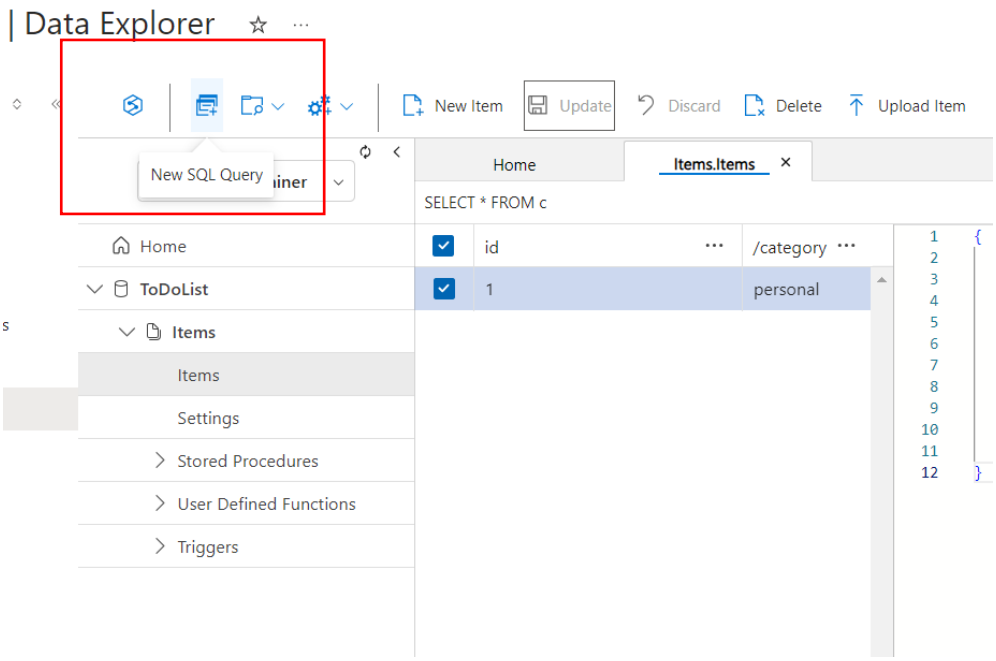
```
{
  "id": "1",
  "category": "personal",
  "name": "groceries",
  "description": "Pick up apples and strawberries.",
  "isComplete": false
}
```

3. Once you've added the json to the **Documents** tab, select **Save**.



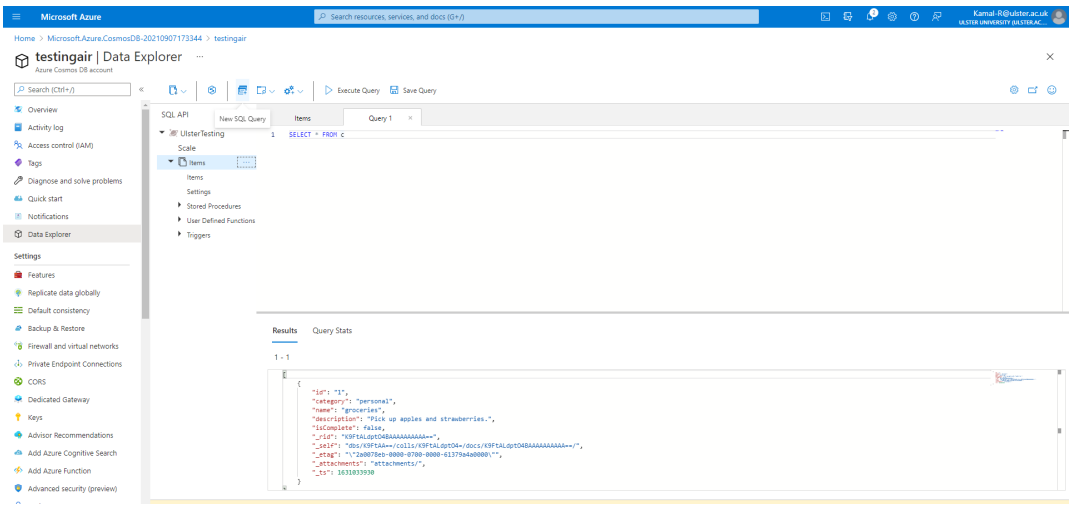
4. Create and save one more document where you insert a unique value for the id property, and change the other properties as you see fit. Your new documents can have any structure you want as Azure Cosmos DB doesn't impose any schema on your data.

Query your data



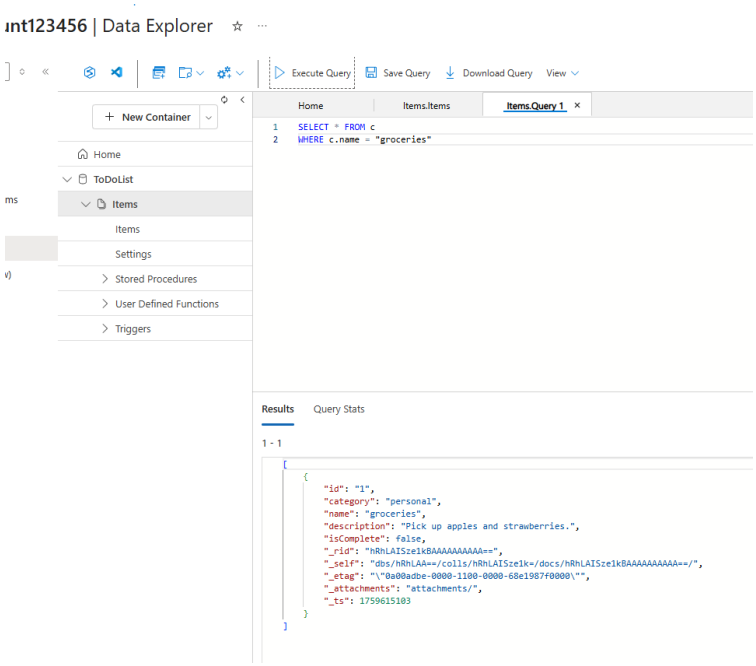
You can use queries in Data Explorer to retrieve and filter your data.

1. At the top of the **Items** tab in Data Explorer, review the default query SELECT * FROM c. This query retrieves and displays all documents from the container ordered by ID.



If you're familiar with SQL syntax, you can enter any supported SQL queries in the query predicate box. You can also use Data Explorer to create stored procedures, UDFs, and triggers for server-side business logic.

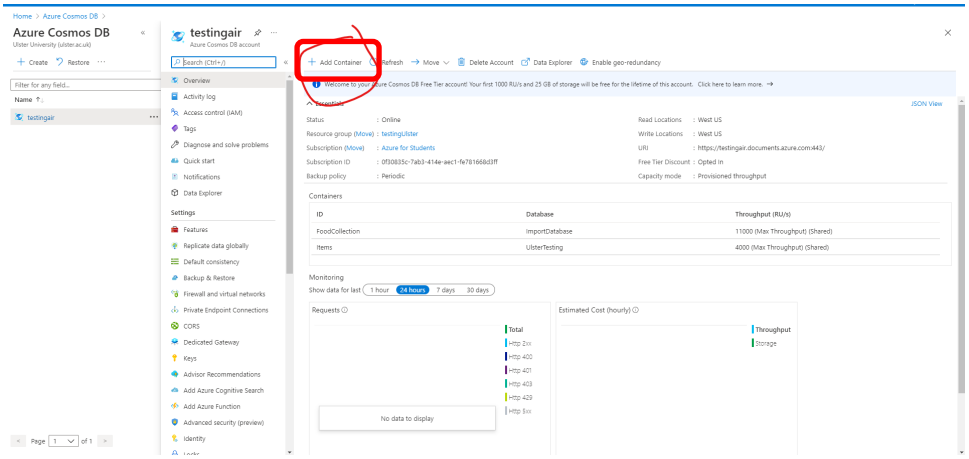
Same queries to extract c.name is below:



Data Explorer provides easy Azure portal access to all of the built-in programmatic data access features available in the APIs. You also use the portal to scale throughput, get keys and connection strings, and review your Azure Cosmos DB account metrics and SLAs.

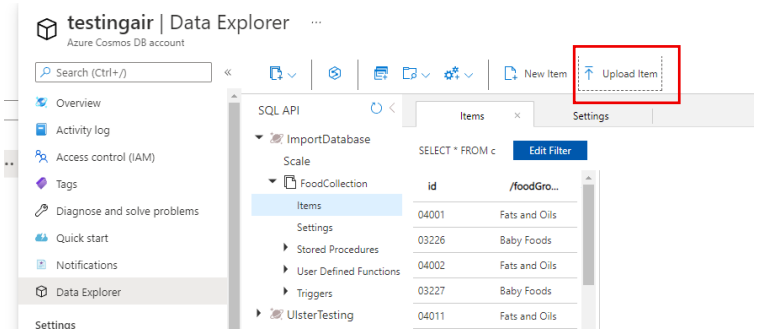
Importing data in Azure Cosmos DB

In this section we are going to upload the json file to Cosmos DB. Go to the your Azure Cosmos DB page.

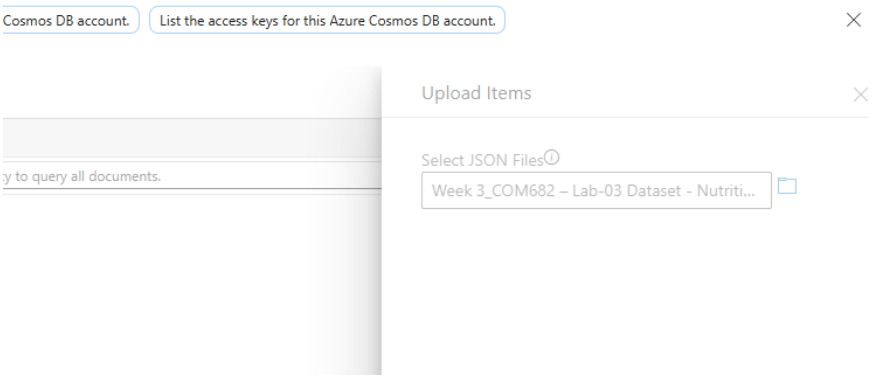


1. In the **Add Container** popup, perform the following actions:
    1. In the **Database id** field, select the **Create new** option and enter the value **ImportDatabase**.
    2. In the **Container Id** field, enter the value **FoodCollection**.
    3. In the **Partition key** field, enter the value /foodGroup.
    4. Select the **OK** button.

Please click on the **ImportDatabase** and click on items under Foodcollection.

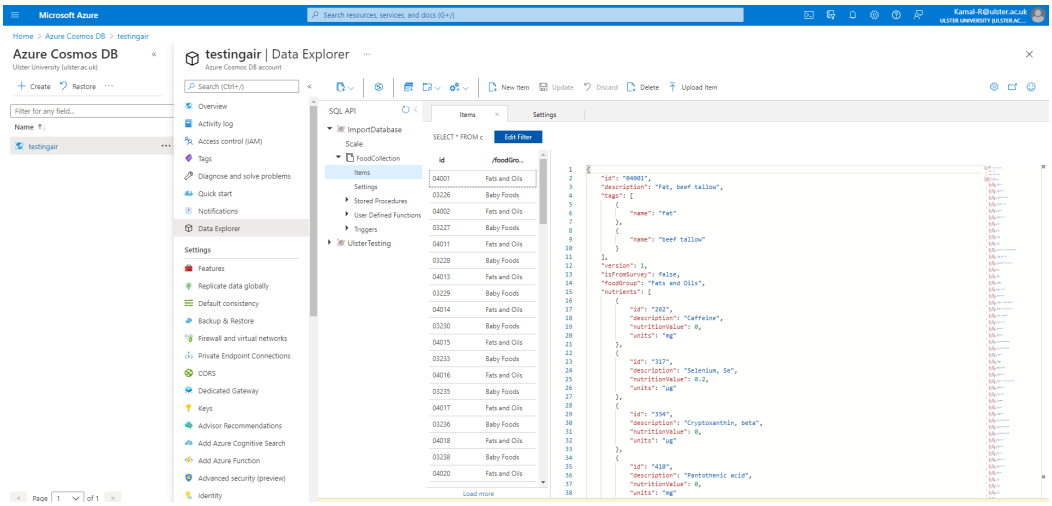


Click on upload item and upload the provided JSON file with this practical. You can find this file on BB with name 'Week-3_COM682_Lab-03 Dataset – NutritionData.json'



Please be patient the file will take time to upload at lease 10 min or more.

After successful upload the data should be look like following snippet.



Azure Cosmos DB SQL API accounts provide support for querying items using the Structured Query Language (SQL), one of the most familiar and popular query languages, as a JSON query language. In this lab, you will explore how to use these rich query capabilities directly through the Azure Portal. No separate tools or client side code are required.

Query Overview

Querying JSON with SQL allows Azure Cosmos DB to combine the advantages of a legacy relational databases with a NoSQL database. You can use many rich query capabilities such as sub-queries or aggregation functions but still retain the many advantages of modeling data in a NoSQL database.

Azure Cosmos DB supports strict JSON items only. The type system and expressions are restricted to deal only with JSON types. For more information, see the JSON specification.

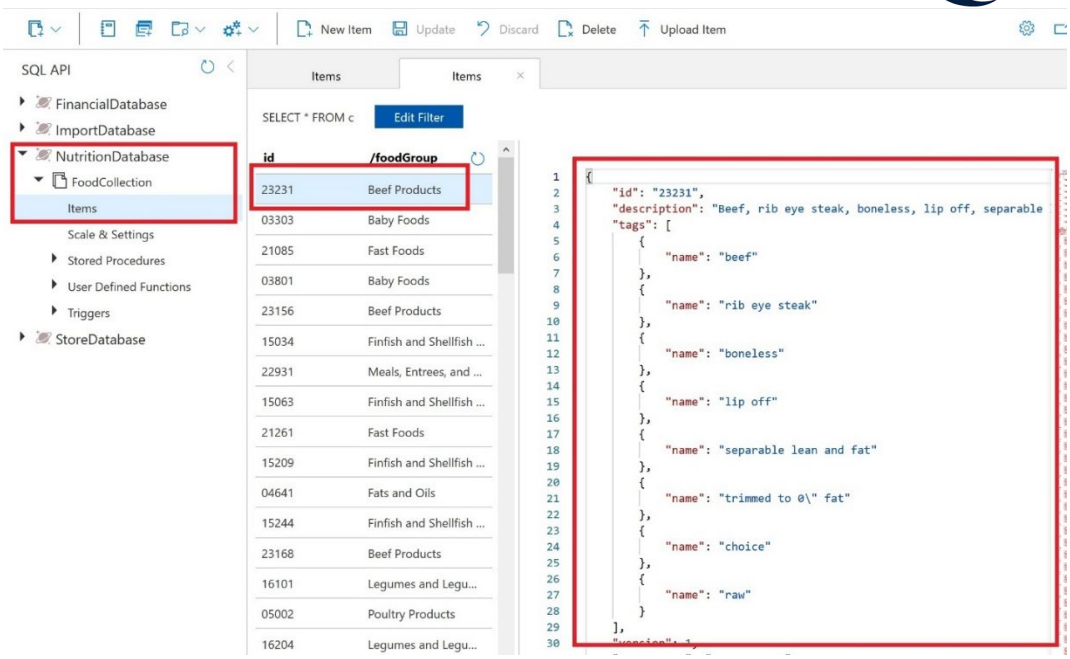
Running your first query

In this lab section, you will query your **FoodCollection**.

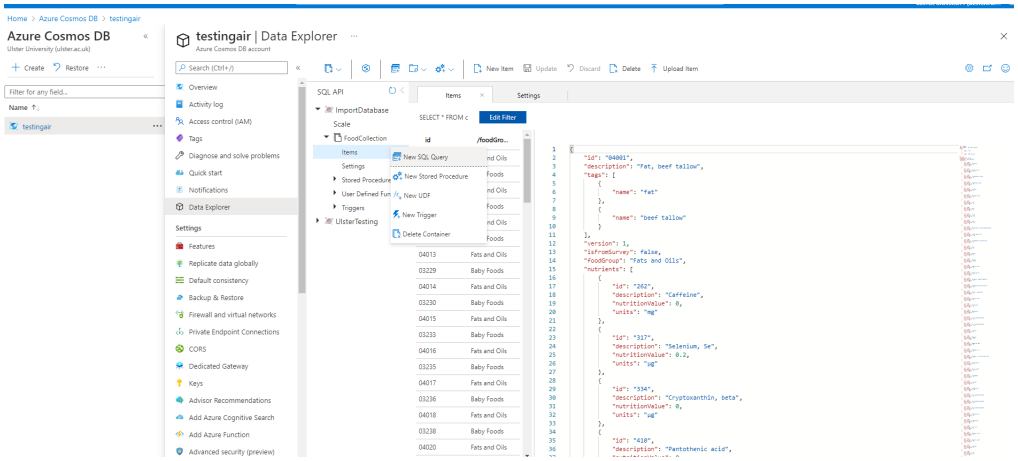
You will begin by running basic queries with SELECT, WHERE, and FROM clauses.

Open Data Explorer

1. Locate and select the **Data Explorer** link on the left side of the page.
2. In the **Data Explorer** section, expand the **NutritionDatabase** database node and then expand the **FoodCollection** container node.
3. Within the **FoodCollection** node, select the **Items** link.
4. View the items within the container. Observe how these documents have many properties, including arrays.

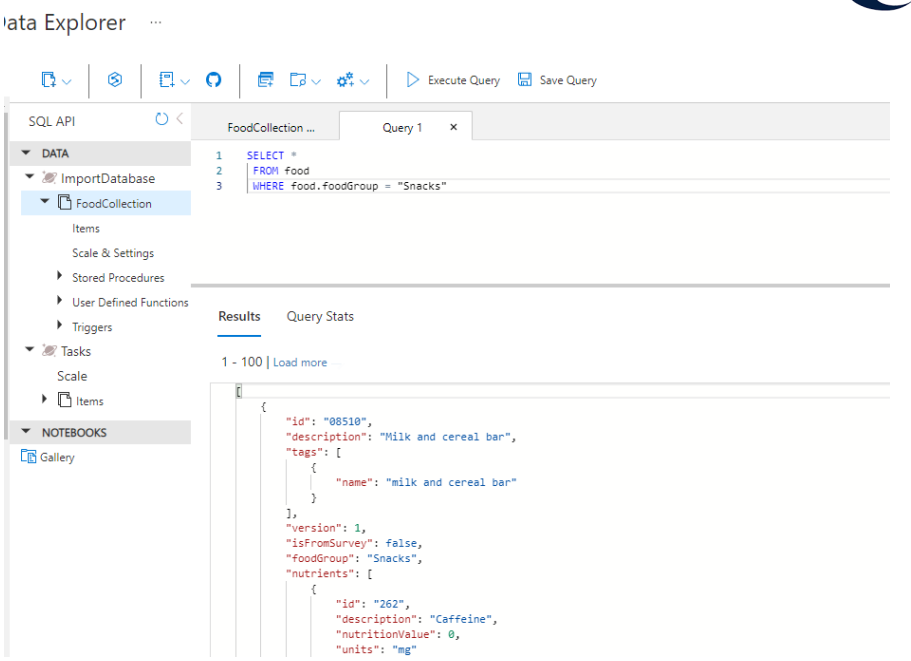


5. Select **New SQL Query**.



6. Paste the following SQL query and select **Execute Query**.

```
SELECT *
FROM food
WHERE food.foodGroup = "Snacks"
```



7. You will see that the query returned the single document foodGroup is “Snacks”. Explore the structure of this item as it is representative of the items within the **FoodCollection** container that we will be working with for the remainder of this section.

Dot and quoted property projection accessors

You can choose which properties of the document to project into the result using the dot notation.

Select **New SQL Query**. Paste the following SQL query and selecting **Execute Query**.

```
SELECT food.id
FROM food
WHERE food.foodGroup = "Snacks"
```

Though less common, you can also access properties using the quoted property operator [””]. For example, SELECT food.id and SELECT food[“id”] are equivalent. This syntax is useful to escape a property that contains spaces, special characters, or has the same name as a SQL keyword or reserved word.

```
SELECT food["id"]
FROM food
WHERE food["foodGroup"] = "Snacks"
```

WHERE clauses

Let's explore WHERE clauses. You can add complex scalar expressions including arithmetic, comparison and logical operators in the WHERE clause.

Run the below query by selecting the **New SQL Query**.

Paste the following SQL query and then select **Execute Query**.

```
SELECT food.id,
food.description,
food.tags,
food.foodGroup,
food.version
FROM food
WHERE (food.foodGroup = "Baby Foods")
```

This query will return the id, description, servings, tags, foodGroup, manufacturerName and version for items with "Baby Foods" for **foodGroup** and a **version** greater than 0.

You should note that where the query returned the results of tags node it projected the entire contents of the property which in this case is an array.

Advanced projection

Azure Cosmos DB supports several forms of transformation on the resultant JSON. One of the simplest is to alias your JSON elements using the AS aliasing keyword as you project your results.

By running the query below you will see that the element names are transformed. In addition, the projection is accessing only the first element in the servings array for all items specified by the WHERE clause.

Run the below query by selecting the **New SQL Query**.

Paste the following SQL query and then select **Execute Query**.

```
SELECT food.description,
food.foodGroup,
food.servings[0].description AS servingDescription,
food.servings[0].weightInGrams AS servingWeight
FROM food
WHERE food.foodGroup = "Fruits and Fruit Juices"
AND food.servings[0].description = "cup"
```

ORDER BY clause

Azure Cosmos DB supports adding an ORDER BY clause to sort results based on one or more properties

Run the below query by selecting the **New SQL Query**.

Paste the following SQL query and then select **Execute Query**.

```
SELECT food.description,
food.foodGroup,
food.servings[0].description AS servingDescription,
food.servings[0].weightInGrams AS servingWeight
FROM food
WHERE food.foodGroup = "Fruits and Fruit Juices" AND food.servings[0].description = "cup"
ORDER BY food.servings[0].weightInGrams DESC
```

You can learn more about configuring the required indexes for an Order By clause in the later Indexing Lab or by reading our docs.

Limiting query result size

Azure Cosmos DB supports the TOP keyword. TOP can be used to limit the number of returning values from a query.

Run the query below to see the top 20 results.

```
SELECT TOP 20 food.id,
food.description,
food.tags,
food.foodGroup
FROM food
WHERE food.foodGroup = "Snacks"
```

The OFFSET LIMIT clause is an optional clause to skip then take some number of values from the query. The OFFSET count and the LIMIT count are required in the OFFSET LIMIT clause.

```
SELECT food.id,
food.description,
food.tags,
food.foodGroup
FROM food
WHERE food.foodGroup = "Snacks"
ORDER BY food.id
OFFSET 10 LIMIT 10
```

When OFFSET LIMIT is used in conjunction with an ORDER BY clause, the result set is produced by doing skip and take on the ordered values. If no ORDER BY clause is used, it will result in a deterministic order of values.

More advanced filtering

Let's add the IN and BETWEEN keywords into our queries. IN can be used to check whether a specified value matches any element in a given list and BETWEEN can be used to run queries against a range of values.

Run the query below:

```
SELECT food.id,
    food.description,
    food.tags,
    food.foodGroup,
    food.version
FROM food
WHERE food.foodGroup IN ("Poultry Products", "Sausages and Luncheon Meats")
  AND (food.id BETWEEN "05740" AND "07050")
```

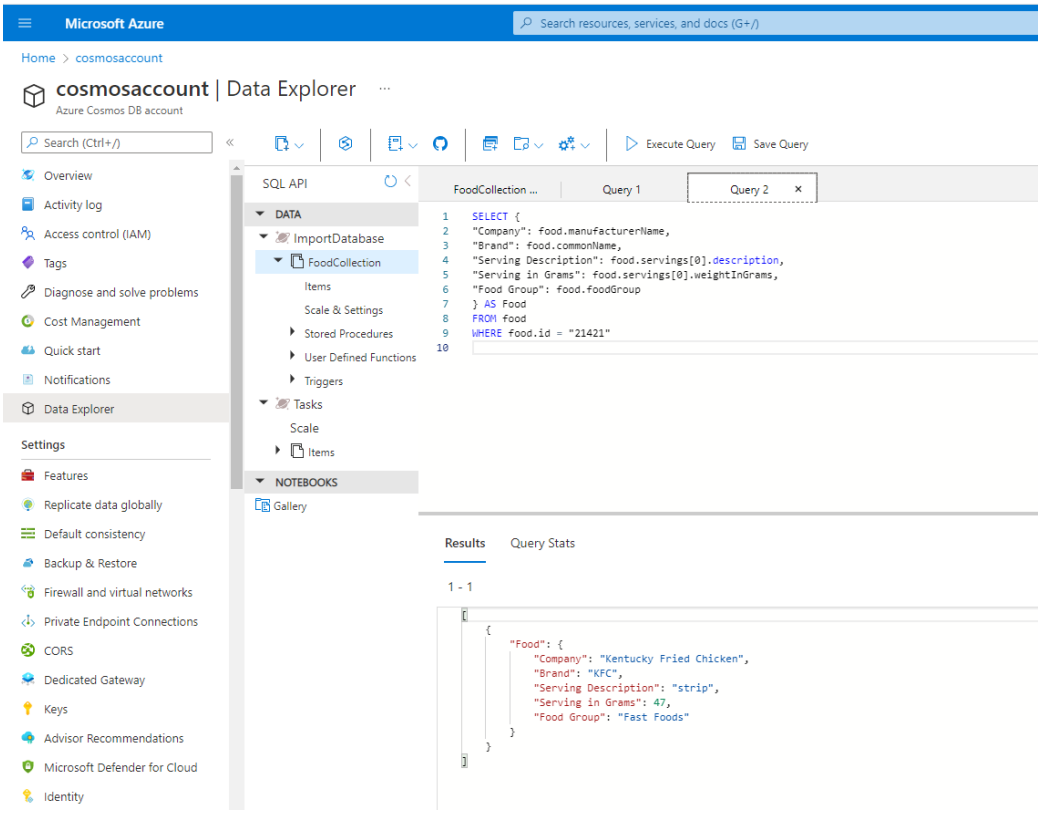
More advanced projection

Azure Cosmos DB supports JSON projection within its queries. Let's project a new JSON Object with modified property names.

Run the query below to see the results.

```
SELECT {
"Company": food.manufacturerName,
"Brand": food.commonName,
"Serving Description": food.servings[0].description,
"Serving in Grams": food.servings[0].weightInGrams,
"Food Group": food.foodGroup
} AS Food
FROM food
```

```
WHERE food.id = "21421"
```



JOIN within your documents

Azure Cosmos DB's JOIN supports intra-document and self-joins. Azure Cosmos DB does not support JOINs across documents or containers.

In an earlier query example we returned a result with attributes of just the first serving of the food.servings array. By using the join syntax below, we can now return an item in the result for every item within the serving array while still being able to project the attributes from elsewhere in the item.

Run the query below to iterate on the food document's servings.

```
SELECT
food.id as FoodID,
serving.description as ServingDescription
FROM food
JOIN serving IN food.servings
WHERE food.id = "03226"
```

JOINs are useful if you need to filter on properties within an array. Run the below example that has filter after the intra-document JOIN.

```
SELECT VALUE COUNT(1)
FROM c
JOIN t IN c.tags
JOIN s IN c.servings
WHERE t.name = 'infant formula' AND s.amount > 1
```

System functions

Azure Cosmos DB supports a number of built-in functions for common operations. They cover mathematical functions like ABS, FLOOR and ROUND and type checking functions like IS_ARRAY, IS_BOOL and IS_DEFINED. Learn more about supported system functions.

Run the query below to see example use of some system functions

```
SELECT food.id,
food.commonName,
food.foodGroup,
ROUND(nutrient.nutritionValue) AS amount,
nutrient.units
FROM food JOIN nutrient IN food.nutrients
```

```
WHERE IS_DEFINED(food.commonName)
AND nutrient.description = "Water"
AND food.foodGroup IN ("Sausages and Luncheon Meats", "Legumes and Legume Products")
```

Correlated sub-queries

In many scenarios, a sub-query may be effective. A correlated sub-query is a query that references values from an outer query. We will walk through some of the most useful examples here. You can learn more about subqueries.

There are two types of sub-queries: Multi-value sub-queries and scalar sub-queries. Multi-value sub-queries return a set of documents and are always used within the FROM clause. A scalar sub-query expression is a sub-query that evaluates to a single value.

Multi-value subqueries

You can optimize JOIN expressions with a sub-query.

Consider the following query which performs a self-join and then applies a filter on name, nutritionValue, and amount. We can use a sub-query to filter out the joined array items before joining with the next expression.

```
SELECT VALUE COUNT(1)
FROM c
JOIN t IN c.tags
JOIN n IN c.nutrients
JOIN s IN c.servings
WHERE t.name = 'infant formula' AND (n.nutritionValue > 0
AND n.nutritionValue < 10) AND s.amount > 1
```

We could rewrite this query using three sub-queries to optimize and reduce the Request Unit (RU) charge. Observe that the multi-value sub-query always appears in the FROM clause of the outer query.

```
SELECT VALUE COUNT(1)
FROM c
JOIN (SELECT VALUE t FROM t IN c.tags WHERE t.name = 'infant formula')
JOIN (SELECT VALUE n FROM n IN c.nutrients WHERE n.nutritionValue > 0 AND n.nutritionValue < 10)
JOIN (SELECT VALUE s FROM s IN c.servings WHERE s.amount > 1)
```

Scalar sub-queries

One use case of scalar sub-queries is rewriting ARRAY_CONTAINS as EXISTS.

Consider the following query that uses ARRAY_CONTAINS:

```
SELECT TOP 5 f.id, f.tags
FROM food f
WHERE ARRAY_CONTAINS(f.tags, {name: 'orange'})
```

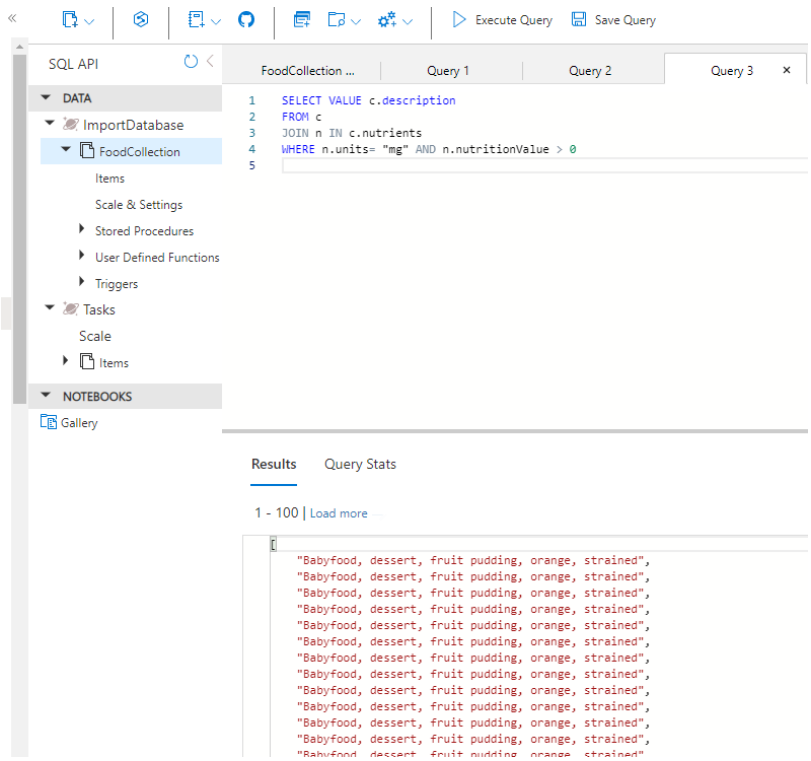
Run the following query which has the same results but uses EXISTS:

```
SELECT TOP 5 f.id, f.tags
FROM food f
WHERE EXISTS(SELECT VALUE t FROM t IN f.tags WHERE t.name = 'orange')
```

The major advantage of using EXISTS is the ability to have complex filters in the EXISTS function, rather than just the simple equality filters which ARRAY_CONTAINS permits.

Here is an example:

```
SELECT VALUE c.description
FROM c
JOIN n IN c.nutrients
WHERE n.units= "mg" AND n.nutritionValue > 0
```



Task 1:

Write in your own words the difference between SQL and NoSQL?

Task 2:

Read the suggested documentation for CosmosDB with the goal of understanding a) its APIs and b) how to integrate it into the web development language of your choice. This understanding will provide a good foundation for developing PaaS applications.

General Cosmos Documentation:

https://docs.microsoft.com/en-us/azure/cosmos-db/

Azure Learn - Choose the appropriate API for Azure Cosmos DB:

https://docs.microsoft.com/en-us/learn/modules/choose-api-for-cosmos-db/1-introduction

Azure Learn - Introduction to Azure Cosmos DB SQL API:

https://docs.microsoft.com/en-us/learn/modules/intro-to-azure-cosmos-db-core-api/

NOTE: If Free tier or Serverless is not selected then large costs may be incurred.

Serverless consumes credit in a miniscule manner. Alternatively you may configure this within the free tier limitations (recommended).

At the time of writing this was 400RU/s provisioned throughput with 25GB of storage. Check your free tier allowance.